Informe Intermedio de Avance

Proyecto: Detección de espacios vacíos en góndolas (CVW Inventory Monitor)
Fecha: Octubre 2026
Integrantes: David Salvatierra, David Yáñez, Francisco Fernández y Francisco Merino.

1. Estado de Avance del Proyecto

El proyecto ha completado con éxito la fase de estructuración base y entrenamiento inicial. Hasta la fecha, se ha desarrollado un modelo de visión computacional utilizando la arquitectura YOLOv8s, diseñado específicamente para detectar dos clases en imágenes de góndolas: product (producto presente) y missing (espacio vacío). Este núcleo de inteligencia artificial ha sido integrado dentro de un sistema modular completo que abarca un backend en FastAPI, un frontend interactivo en React, y servicios desacoplados que gestionan la cámara, la inferencia en tiempo real, el cálculo de inventario y la emisión de alertas.

Hallazgos, Desafíos y Riesgos Identificados

Durante las iteraciones preliminares, se implementó un pipeline de aumentación de datos (geométrica y de degradación) para simular condiciones reales de hardware de seguridad (baja resolución, compresión JPEG, ruido). Esto permitió identificar varios desafíos críticos. En primer lugar, se detectó un severo desbalance de clases: la clase crítica missing es aproximadamente 37 veces menos frecuente que la clase product. Adicionalmente, el análisis dimensional reveló que un 5% de los productos mide menos de 7.7 píxeles a una resolución de 640 px, situándolos por debajo de la capacidad de detección de la cuadrícula base de YOLO (stride mínimo de 8 px).

Riesgo de Data Leakage Mitigado:

Al procesar imágenes casi idénticas de las mismas góndolas, un split aleatorio tradicional inflaría artificialmente las métricas. Este riesgo fue mitigado de forma efectiva implementando hashes perceptuales para agrupar imágenes visualmente similares antes de ejecutar la partición del dataset.

Riesgo de Salto de Dominio:

El modelo se entrenó y evaluó con fotografías de góndolas, no con video de cámaras de seguridad reales de tienda. La partición "test cámara" simula esa condición de forma sintética (compresión, baja resolución, ruido, sombras y escala de grises), por lo que el desempeño en sala debe validarse con imágenes reales de las cámaras antes del despliegue.

2. Documentación y Arquitectura de la Solución

La estrategia técnica seleccionada sitúa a un modelo YOLOv8s (aproximadamente 11 millones de parámetros) como el motor de inferencia principal, operando sobre imágenes de 640x640 píxeles. La justificación de esta arquitectura radica en la eficiencia You Only Look Once, que permite realizar la detección en una sola pasada de red neuronal, siendo ideal para un procesamiento ágil. Se escaló desde la versión "nano" (YOLOv8n) debido a que esta se estancaba en un mAP50-95 de 0.38, mientras que modelos superiores tendían a memorizar el conjunto de datos limitado.

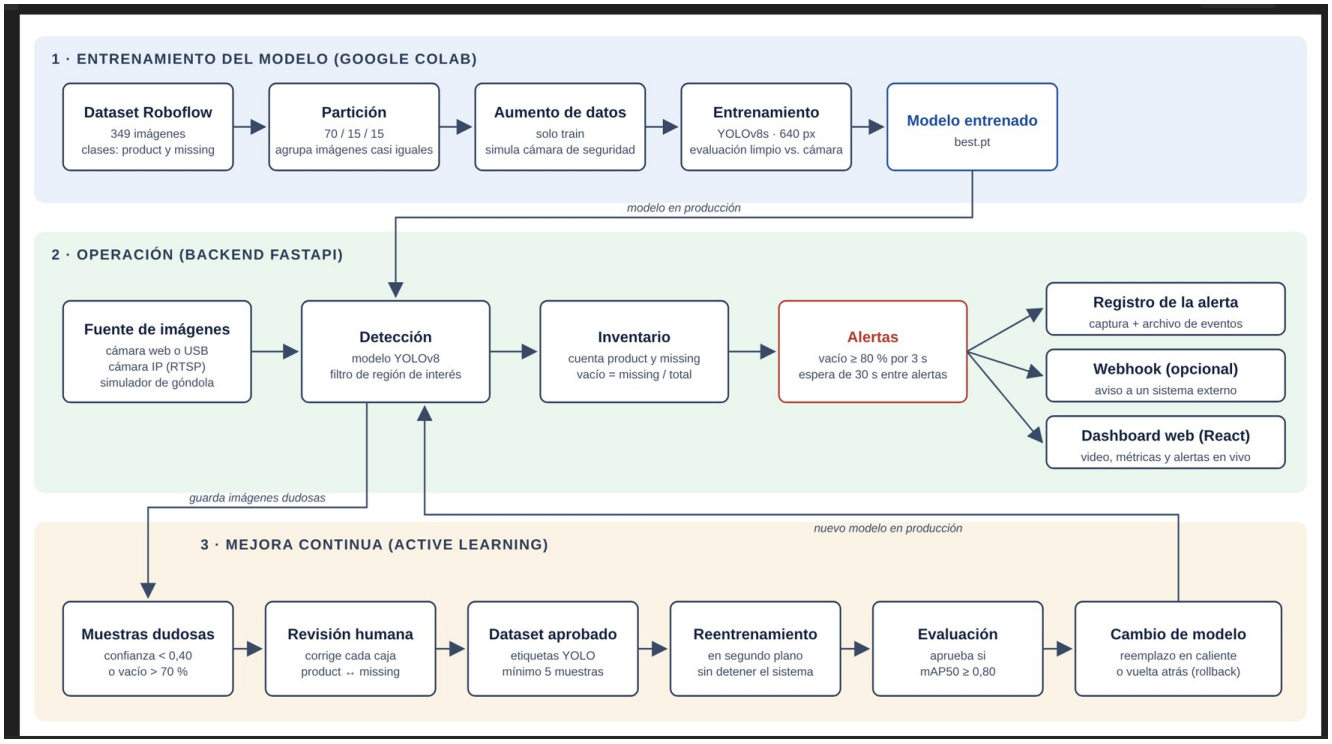
Flujo de Operación

El flujo en producción comienza con la captura del stream de video mediante el servicio de cámara. Este pasa por un filtro de Región de Interés (ROI) para descartar pasillos y techos, alimentando el módulo de inventario. Si la tasa de espacios vacíos calculada alcanza o supera el 80% durante una ventana temporal sostenida de 3 segundos, se dispara una alerta. Este mecanismo previene falsos positivos, y va acompañado de un periodo de enfriamiento (cooldown) de 30 segundos.

Datasets y Preparación de Datos

El dataset base, proveniente de Roboflow, consta de 349 imágenes (versión 4 del dataset). Para garantizar la generalización del modelo y simular de manera fiel el salto de dominio esperado en producción, los datos fueron rigurosamente estructurados:

Partición	Porcentaje	Características y Tratamiento	Propósito Principal
Train	70%	Imágenes base con aumentación geométrica intensiva y filtros de color.	Ajuste de pesos de la red neuronal.
Valid / Test	15% / 15%	Imágenes limpias, sin alteraciones visuales, para métricas base.	Selección de modelo y benchmark ideal.
Valid_cam / Test_cam	Copias	Simulación de cámara: compresión extrema, baja resolución, ruido.	Evaluación de rendimiento en entorno real.



Ajustes respecto a la propuesta inicial.

Durante el desarrollo, la solución evolucionó en los siguientes puntos:

- **Modelo:** se pasó de YOLOv8n a YOLOv8s, porque la versión nano se estancó en un mAP50-95 de 0,38 y se espera incorporar más imágenes al dataset.
- **Fuente de imágenes:** el procesamiento por lotes de imágenes pregrabadas se reemplazó por un servicio de cámara que acepta cámara web, cámara IP (RTSP) o un simulador de

góndola, para acercar la solución a las condiciones de las cámaras de seguridad de las salas.
- **Regla de alerta:** el semáforo de tres niveles se simplificó al umbral crítico: se alerta cuando el espacio vacío llega al 80 % o más, lo que equivale a un 20 % o menos de stock visible.
- **Notificación:** cada alerta queda registrada con una captura y un archivo de eventos, y puede enviarse a un sistema externo mediante un *webhook*. [Completar: canal definitivo de notificación.]

3. Análisis de Resultados Preliminares

Las pruebas iniciales sobre las particiones degradadas muestran un comportamiento prometedor. A pesar del fuerte desbalance en la información de entrenamiento, el modelo ha demostrado una robustez considerable frente a la perspectiva angular típica de cámaras cenitales y los artefactos de compresión.

Métricas del modelo (YOLOv8s, versión 2)

Las métricas siguientes se obtuvieron sobre la partición de test (53 imágenes que el modelo nunca vio en entrenamiento) y sobre su copia degradada "test cámara". El modelo fue YOLOv8s entrenado durante 0,76 h en una GPU T4 de Google Colab.

Métrica	Test (imágenes limpias)	Test cámara (degradado)
Precisión	69,2 %	63,5 %
Recall	62,9 %	56,8 %
mAP50	63,3 %	58,2 %
mAP50-95	33,6 %	28,1 %
mAP50 clase missing (vacío)	45,6 %	43,8 %
Recall clase missing	47,9 %	43,7 %
mAP50 clase product	81,0 %	72,7 %
Recall clase product	77,9 %	70,0 %

En validación el modelo alcanzó mAP50 = 68,0 % y mAP50-95 = 37,5 %. La degradación tipo cámara resta 5,5 puntos de mAP50 en test (5,7 en validación). La clase missing, que es la que dispara las alertas, es la más débil: el modelo encuentra menos de la mitad de los espacios vacíos (recall 47,9 % en test y 43,7 % en test cámara). El umbral de confianza recomendado por la curva F1 es 0,38. Estas cifras no son directamente comparables con la versión 1 (YOLOv8n, ~199 imágenes, partición aleatoria, mAP50 de 72,8 % en validación), ya que la versión 2 usa un dataset mayor y una partición agrupada más estricta.

Análisis de Errores y Limitaciones

Mejores y peores casos (partición test cámara)

Para elegir los casos se evaluó cada una de las 53 imágenes de test cámara con el umbral de confianza 0,38. Cada detección del modelo se emparejó uno a uno con una caja real de la misma clase (IoU ≥ 0,5); así se cuentan por imagen los aciertos, los falsos positivos y los objetos no detectados de cada clase, y se

calcula su F1. El puntaje de la imagen es el promedio del F1 de las clases presentes (1 = perfecto, 0 = nada correcto).

Cómo leer las figuras: panel izquierdo = cajas reales del etiquetado; panel derecho = predicciones del modelo. Verde = product (producto) y rojo = missing (espacio vacío).

Mejores casos



Mejor caso 1 — imagen 57 — puntaje 0,978. Productos detectados 178 de 187 (falsos positivos 7); vacíos detectados 1 de 1, falsas alarmas 0. Góndola frontal, bien iluminada y que llena el cuadro. Detecta casi todos los productos y el único espacio vacío.



Mejor caso 2 — imagen 490 — puntaje 0,924. Productos detectados 112 de 116 (falsos positivos 36); vacíos detectados 2 de 2, falsas alarmas 0. Vista frontal con huecos oscuros bien definidos: detecta los 2 espacios vacíos. Tiene 36 falsos positivos de product, que bajan su precisión.



Mejor caso 3 — imagen train_4679 — puntaje 0,885. Productos detectados 132 de 146 (falsos positivos 11); vacíos detectados 6 de 8, falsas alarmas 0. Filas ordenadas y huecos visibles: detecta 6 de 8 espacios vacíos sin falsas alarmas de vacío.



Mejor caso 4 — imagen 1132 — puntaje 0,858. Productos detectados 130 de 154 (falsos positivos 19); sin vacíos etiquetados, falsas alarmas 0. Góndola completa sin espacios vacíos etiquetados; buen conteo de productos pese a la degradación.



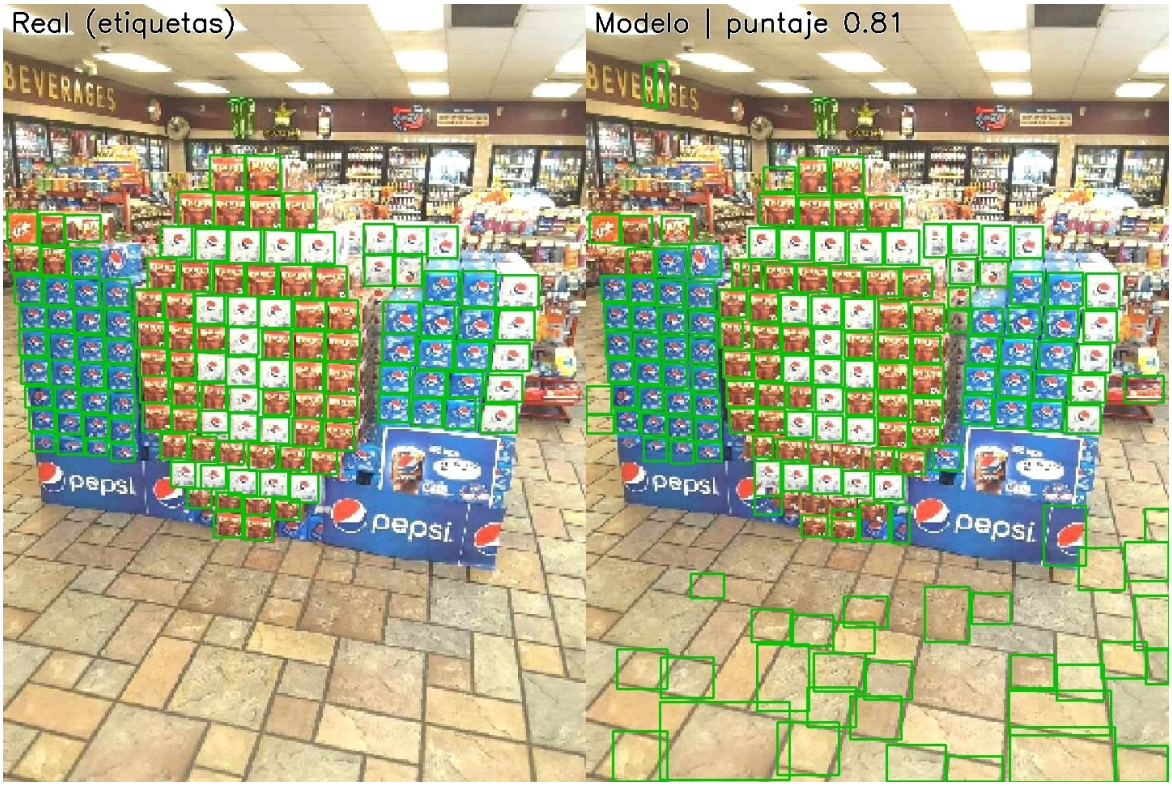
Mejor caso 5 — imagen train_4861 — puntaje 0,849. Productos detectados 104 de 111 (falsos positivos 30); sin vacíos etiquetados, falsas alarmas 0. Góndola sin vacíos etiquetados; detecta 104 de 111 productos. Tiene 30 falsos positivos de product.



Mejor caso 6 — imagen 463 — puntaje 0,820. Productos detectados 115 de 121 (falsos positivos 28); vacíos detectados 5 de 5, falsas alarmas 3. Detecta los 5 espacios vacíos, pero agrega 3 falsas alarmas de vacío en zonas oscuras entre productos.



Mejor caso 7 — imagen train_4741 — puntaje 0,812. Productos detectados 170 de 215 (falsos positivos 28); vacíos detectados 4 de 6, falsas alarmas 0. Góndola grande y densa: detecta 4 de 6 vacíos sin falsas alarmas; quedan 45 productos sin detectar.



Mejor caso 8 — imagen 708 — puntaje 0,805. Productos detectados 118 de 127 (falsos positivos 48); sin vacíos etiquetados, falsas alarmas 0. Exhibidor de piso de bebidas. Atención: el modelo marca falsos "product" sobre las baldosas del piso (48 falsos positivos). Queda arriba en el ranking solo porque la imagen no tiene vacíos etiquetados.



Mejor caso 9 — imagen 1025 — puntaje 0,799. Productos detectados 130 de 144 (falsos positivos 5); vacíos detectados 1 de 1, falsas alarmas 1. Muy buena precisión en productos (solo 5 falsos positivos) y detecta el espacio vacío, con 1 falsa alarma.



Mejor caso 10 — imagen train_3619 — puntaje 0,786. Productos detectados 116 de 163 (falsos positivos 16); sin vacíos etiquetados, falsas alarmas 0. Pese a la sombra sintética aplicada, mantiene buena precisión; quedan 47 productos sin detectar.

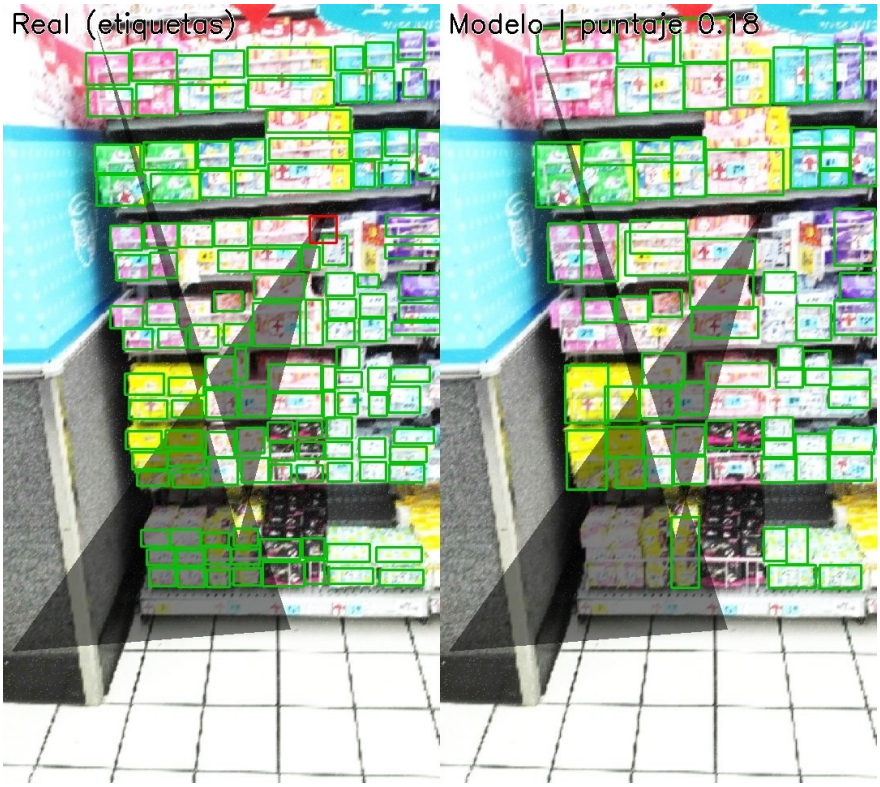
Peores casos



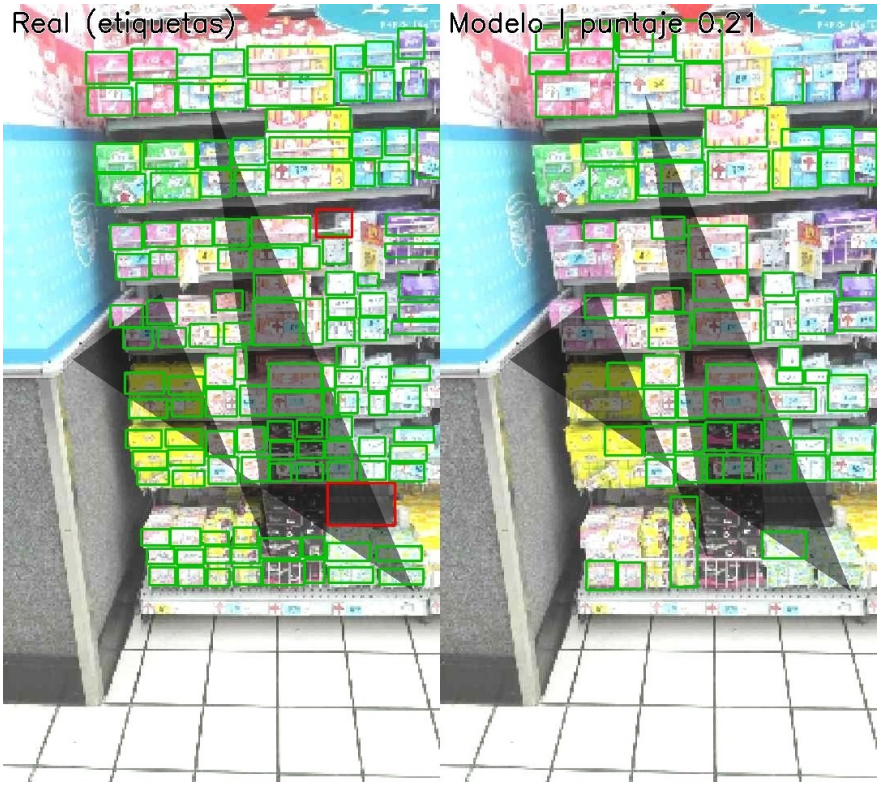
Peor caso 1 — imagen 1421 — puntaje 0,165. Productos detectados 40 de 173 (falsos positivos 30); sin vacíos etiquetados, falsas alarmas 2. Las sombras sintéticas (triángulos negros) sobre los envases confunden al modelo: solo detecta 40 de 173 productos y genera 2 falsas alarmas de vacío.



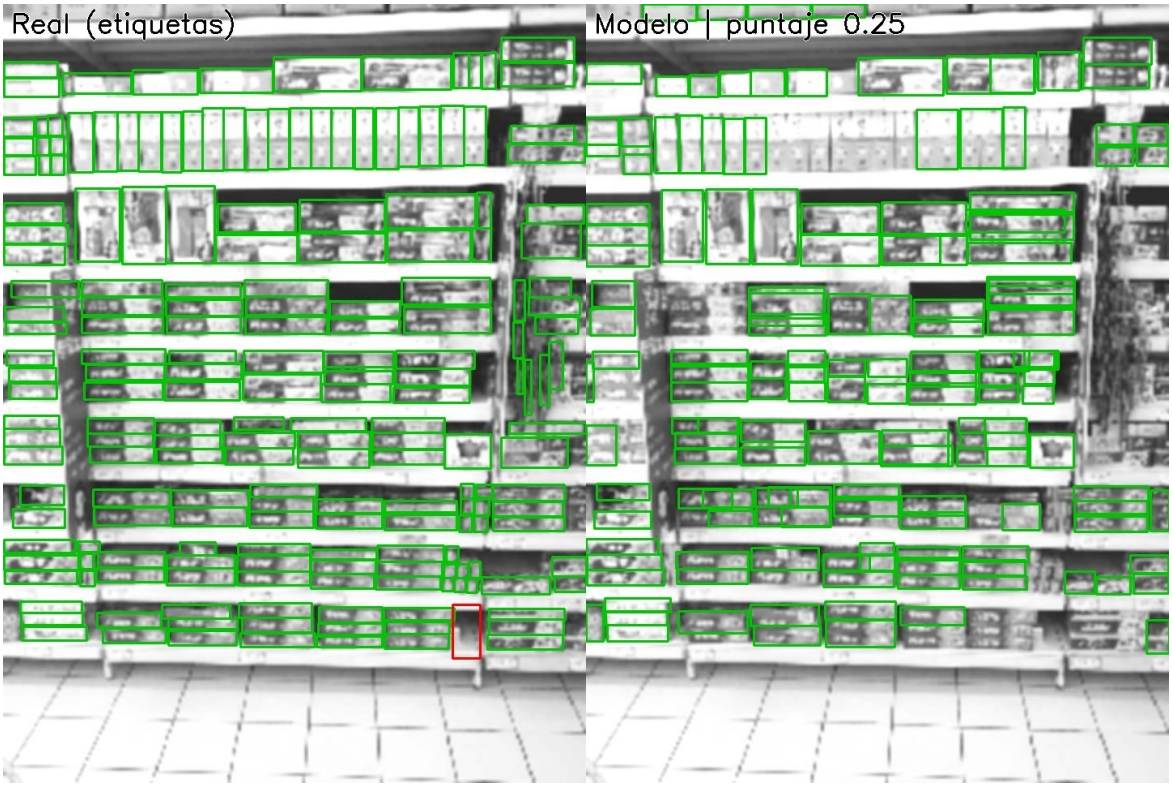
Peor caso 2 — imagen train_4026 — puntaje 0,177. Productos detectados 79 de 320 (falsos positivos 47); vacíos detectados 0 de 2, falsas alarmas 1. Góndola extremadamente densa con productos muy pequeños: detecta 79 de 320 productos y ninguno de los 2 espacios vacíos.



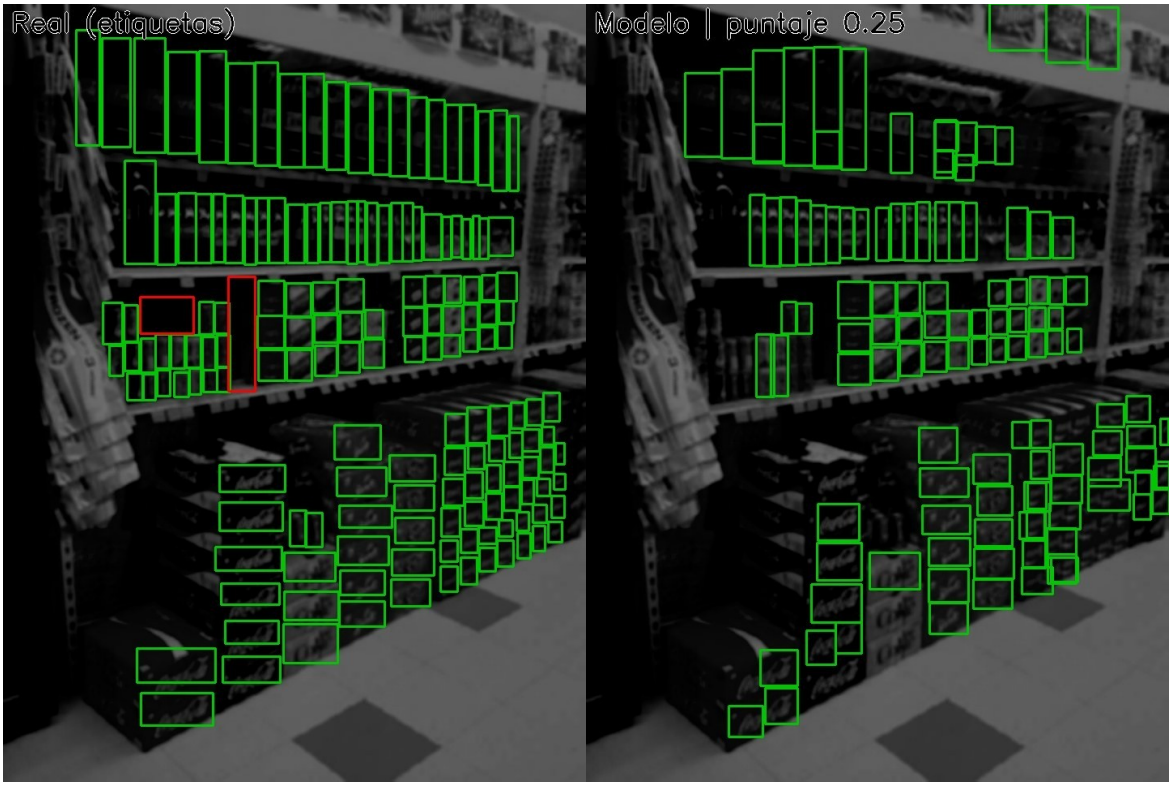
Peor caso 3 — imagen train_4683 — puntaje 0,184. Productos detectados 37 de 127 (falsos positivos 37); vacíos detectados 0 de 1, falsas alarmas 0. Cabecera con sombras sintéticas y piso blanco reflectante: no detecta el espacio vacío y pierde 90 productos.



Peor caso 4 — imagen train_4683 — puntaje 0,214. Productos detectados 42 de 127 (falsos positivos 27); vacíos detectados 0 de 2, falsas alarmas 0. Misma fotografía que el caso anterior (otra exportación de Roboflow): no detecta los 2 vacíos. Confirma que el problema es la escena y no un error puntual.



Peor caso 5 — imagen 995 — puntaje 0,246. Productos detectados 85 de 193 (falsos positivos 68); vacíos detectados 0 de 1, falsas alarmas 0. Imagen en escala de grises (simulación de cámara monocromática/IR): sin color, el fondo de la góndola se confunde con los envases y el vacío no se detecta.



Peor caso 6 — imagen 1459 — puntaje 0,253. Productos detectados 71 de 162 (falsos positivos 48); vacíos detectados 0 de 2, falsas alarmas 0. Escala de grises, muy oscura y en ángulo oblicuo: no detecta ninguno de los 2 espacios vacíos.



Peor caso 7 — imagen 319 — puntaje 0,312. Productos detectados 131 de 210 (falsos positivos 79); vacíos detectados 0 de 6, falsas alarmas 0. Vista frontal, pero los huecos son pequeños y oscuros: no detecta ninguno de los 6 vacíos y genera 79 falsos positivos de product.



Peor caso 8 — imagen train_4641 — puntaje 0,336. Productos detectados 112 de 181 (falsos positivos 40); vacíos detectados 0 de 9, falsas alarmas 0. Góndola de envases rojos: los huecos quedan entre productos de color intenso y el modelo no detecta ninguno de los 9 vacíos.



Peor caso 9 — imagen train_4101 — puntaje 0,342. Productos detectados 130 de 228 (falsos positivos 22); vacíos detectados 0 de 6, falsas alarmas 0. Vitrina refrigerada vista de lejos: no detecta los 6 espacios vacíos.



Peor caso 10 — imagen train_4302 — puntaje 0,344. Productos detectados 52 de 210 (falsos positivos 31); vacíos detectados 1 de 3, falsas alarmas 2. Pasillo en ángulo oblicuo con productos lejanos y pequeños: detecta solo 52 de 210 productos y 1 de 3 vacíos.

Patrones observados. Los mejores casos comparten góndolas frontales, bien iluminadas, que ocupan todo el cuadro y con huecos oscuros bien definidos. En los 10 peores casos el modelo detectó solo 1 de los 32 espacios vacíos reales; las causas visibles son sombras sintéticas, imágenes en escala de grises u oscuras, vistas oblicuas o lejanas, vitrinas vistas de lejos y góndolas muy densas con productos pequeños. Es decir, el error dominante es no detectar el vacío (falso negativo), que es justamente el evento que debe generar la alerta.

Al analizar los casos de menor rendimiento, las limitaciones observadas derivan en gran medida de la naturaleza ambigua del objetivo. Un "espacio vacío" carece de bordes definidos o texturas propias, consistiendo meramente en el fondo de la góndola expuesto. Adicionalmente, el fenómeno del tamaño mínimo genera la mayor tasa de falsos negativos: productos físicos que, tras la compresión a la resolución de inferencia de 640x640, ocupan menos de 8 píxeles de lado, quedan matemáticamente fuera del alcance de la cuadrícula de análisis del algoritmo, siendo sistemáticamente ignorados.

Próximos Pasos y Acciones de Mejora

Para mitigar estas limitaciones antes del paso a producción, se ha estructurado un plan de acción técnica centrado en mejorar la resolución de inferencia y evaluar heurísticas avanzadas para segmentación de área:

Acción Propuesta	Descripción Técnica	Impacto Esperado
Aumento de imgsz	Elevar la resolución de entrada en inferencia a 800 o 960 píxeles.	Aumentar el recall en productos pequeños al superar el límite de 8 px.
Implementación SAHI	Inferencia por cuadrantes superpuestos (Slicing Aided Hyper Inference).	Mejorar la detección de objetos densamente agrupados en los estantes.

Experimentos SAM	Pruebas A/B usando Segment Anything Model para refinar recortes.	Determinar si justifica el costo computacional vs ganancias en métrica IoU.

Resultado preliminar de SAM: las máscaras casi no colapsaron (2 %) y se salen poco de la caja (alrededor de 10 %), pero cada imagen tarda unos 500 ms. El resultado es no concluyente: para decidir si SAM mejora la estimación del área vacía se necesitan polígonos dibujados a mano en unas 20 imágenes.

4. Repositorio y Reproducibilidad

El código fuente del proyecto presenta a la fecha una arquitectura de software modular, con componentes separados por responsabilidad. El repositorio organiza lógicamente los componentes en directorios específicos como backend/ (para la API en FastAPI), frontend/ (aplicación React con TypeScript), y zonas de persistencia controlada de estado, tales como models/, data/ y alerts/.

Enlace al Repositorio Oficial:
https://github.com/Linzdhx/Capstone.git
Incluye documentación completa en formato README.md para inicialización local y mediante contenedores Docker.

Ejecución y Testing Automatizado

El pipeline de entrenamiento (notebook de Google Colab, que se incorporará al repositorio) fija semillas de aleatoriedad globales (SEED = 42) garantizando que las particiones y aumentaciones sean idénticas entre ejecuciones. En cuanto a la infraestructura de software, se cuenta con una suite de pruebas construida con pytest que valida la lógica crítica del negocio (protección contra división por cero en cálculos de inventario, filtrado geométrico y ventanas temporales).

Simulación Integrada:
El sistema ha sido diseñado para ejecutarse y ser testeado incluso en ausencia de hardware físico. Configurando la fuente como synthetic, el backend levanta un simulador de góndola virtual, permitiendo al equipo de QA probar los eventos, tiempos de cooldown y disparos de alertas de manera controlada y sin dependencias externas.